

CHAPTER 1 · SUB-CHAPTER 1.8

Exception Handling

Hierarchy · Checked vs Unchecked · try-with-resources · Multi-Catch
Custom Exceptions · finally Semantics · Anti-Patterns · Production Patterns

~20 Pages

52+ Q&A

Adv. Questions

Interview Ready

■ Purple [ADV] questions indicate advanced / senior-level depth

Section 1 — Exception Hierarchy

```
java.lang.Throwable
+-- java.lang.Error (do NOT catch — JVM-level problems)
| +-- OutOfMemoryError
| +-- StackOverflowError
| +-- AssertionError
| +-- LinkageError
| +-- NoClassDefFoundError
| +-- UnsatisfiedLinkError
+-- java.lang.Exception
+-- java.lang.RuntimeException (UNCHECKED — compiler does not require handling)
| +-- NullPointerException
| +-- ArrayIndexOutOfBoundsException
| +-- ClassCastException
| +-- NumberFormatException
| +-- IllegalArgumentException
| | +-- IllegalStateException
| +-- UnsupportedOperationException
| +-- ConcurrentModificationException
| +-- ArithmeticException (/ by zero)
| +-- StackOverflowError (also an Error, not here)
+-- (Checked exceptions — compiler REQUIRES handling or declaration)
+-- java.io.IOException
| +-- FileNotFoundException
| +-- SocketException
+-- java.sql.SQLException
+-- java.lang.ClassNotFoundException
+-- java.lang.InterruptedException
+-- java.lang.ReflectiveOperationException
+-- java.lang.reflect.InvocationTargetException
```

Category	Examples	Must Handle?	Recovery?	JVM Responsibility?
Error	OOM, StackOverflow, LinkageError	No — don't catch	Rarely possible	Yes — JVM-level
Checked Exception	IOException, SQLException, InterruptedException	Yes — or declare throws	Expected recoverable	No — application logic
Unchecked / RuntimeException	NPE, ClassCast, IllegalArg	No — optional	Programming errors	No — application bugs

Section 2 — Checked vs Unchecked: Design Philosophy

2.1 When to Use Each

Use	Checked	Unchecked / RuntimeException
Condition type	Recoverable external failure (file missing, network down)	Programming error / violated contract
Caller action	Caller MUST handle (or explicitly declare)	Caller CAN handle but usually shouldn't
Example	FileNotFoundException, SQLException	NullPointerException, IllegalArgumentException
Coupling	Tight — every caller is coupled to the exception type	Loose — callers don't need to know
API evolution	Hard — adding throws breaks all callers	Easy — can add unchecked silently
Industry trend	Spring, JPA, Hibernate all use unchecked	Checked used in core JDK I/O APIs

2.2 The Checked Exception Debate

```
// CHECKED: Forces caller to handle – good for recoverable conditions
// Example: database connection failure – caller should retry or use fallback
public Connection getConnection() throws SQLException {
    return DriverManager.getConnection(url, user, pass);
}
// Caller must handle:
try {
    conn = getConnection();
} catch (SQLException e) {
    // use fallback data source – MEANINGFUL recovery
    conn = fallbackDataSource.getConnection();
}

// UNCHECKED: Contract violation – caller should not "handle", just fix the bug
// Example: null passed to method that requires non-null
public void process(String data) {
    Objects.requireNonNull(data, "data must not be null"); // throws NPE if null
    // ...
}

// CHECKED-TO-UNCHECKED WRAPPING: Common pattern in enterprise Java
// Wrap low-level checked in domain-specific unchecked
public User findUser(Long id) {
    try {
        return userRepository.findById(id);
    } catch (SQLException e) {
        // Translate: SQL details are an implementation detail
        throw new DataAccessException("Failed to find user: " + id, e);
    }
}
// DataAccessException extends RuntimeException – callers don't need to declare it
// But the cause chain is preserved for debugging
```

Section 3 — try-with-resources & AutoCloseable

3.1 AutoCloseable & Closeable

```
// AutoCloseable (Java 7): close() throws Exception
interface AutoCloseable { void close() throws Exception; }

// Closeable (Java 5, refined in 7): close() throws IOException
```

```
// Closeable extends AutoCloseable; overrides close() to narrow exception
interface Closeable extends AutoCloseable { void close() throws IOException; }

// TWR: resources declared in () are auto-closed in REVERSE order
try (Connection conn = dataSource.getConnection();
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery(sql)) {
    while (rs.next()) { process(rs); }
} // rs.close() first, then stmt.close(), then conn.close() – GUARANTEED

// Equivalent manual try-finally (MUCH more error-prone):
Connection conn = null; Statement stmt = null; ResultSet rs = null;
try {
    conn = dataSource.getConnection();
    stmt = conn.createStatement();
    rs = stmt.executeQuery(sql);
    while (rs.next()) { process(rs); }
} finally {
    if (rs != null) try { rs.close(); } catch (SQLException ignored) {}
    if (stmt != null) try { stmt.close(); } catch (SQLException ignored) {}
    if (conn != null) try { conn.close(); } catch (SQLException ignored) {}
}
```

3.2 Suppressed Exceptions

KEY RULE: If the try body throws AND close() also throws, the body exception is PRIMARY and the close() exception is SUPPRESSED (attached via addSuppressed()). The suppressed exception is NOT lost — retrieve it with getSuppressed().

```
class BuggyResource implements AutoCloseable {
    @Override public void close() throws Exception {
        throw new RuntimeException("close() failed");
    }
}

try (BuggyResource r = new BuggyResource()) {
    throw new RuntimeException("body failed"); // PRIMARY exception
}
// PRIMARY exception: "body failed"
// SUPPRESSED (attached): "close() failed"
// Accessing suppressed exceptions:
try (BuggyResource r = new BuggyResource()) {
    throw new RuntimeException("body failed");
} catch (RuntimeException e) {
    System.out.println("Primary: " + e.getMessage()); // "body failed"
    for (Throwable suppressed : e.getSuppressed()) {
        System.out.println("Suppressed: " + suppressed.getMessage()); // "close() failed"
    }
}

// Custom AutoCloseable – best practices:
class ManagedTransaction implements AutoCloseable {
    private boolean committed = false;
    public void commit() { this.committed = true; /* ... */ }
    @Override public void close() {
        if (!committed) {
            rollback(); // auto-rollback on exception exit
        }
    }
}

try (ManagedTransaction tx = new ManagedTransaction()) {
    updateOrder(orderId);
}
```

```
creditAccount(userId, amount);  
tx.commit(); // only reached if no exception  
} // if exception thrown above, close() auto-rolls back
```

Section 4 — Multi-Catch, Chaining & Stack Traces

4.1 Multi-Catch (Java 7+)

```
// Before Java 7: repeated catch blocks
try { riskyOperation(); }
catch (IOException e) { log(e); throw new ServiceException(e); }
catch (SQLException e) { log(e); throw new ServiceException(e); }

// Java 7+ multi-catch: union type – cleaner, no duplication
try { riskyOperation(); }
catch (IOException | SQLException e) {
    log(e);
    throw new ServiceException("Data access failed", e);
}

// RULES for multi-catch:
// 1. Exception types in union must be UNRELATED (not in same hierarchy)
// 2. catch (IOException | FileNotFoundException e) – COMPILE ERROR:
//    FileNotFoundException IS-A IOException – redundant
// 3. The caught variable is EFFECTIVELY FINAL in multi-catch block
// (you can't reassign e in the catch block)
// Ordering of separate catch blocks matters:
try { riskyOp(); }
catch (FileNotFoundException e) { /* specific */ } // subclass first
catch (IOException e) { /* general */ } // superclass after
// catch (Exception e) { } THEN catch (IOException e) { } -- compile error!
```

4.2 Exception Chaining (Wrapping)

```
// Preserving the cause chain is CRITICAL for debugging
// Pattern 1: Constructor with cause (preferred)
try {
    return jdbcTemplate.queryForObject(sql, User.class);
} catch (DataAccessException e) {
    throw new UserServiceException("Failed to load user id=" + id, e); // cause preserved
}

// Pattern 2: initCause() – when cause is known after construction
Exception wrapper = new ServiceException("wrapped");
wrapper.initCause(originalCause); // can only call once; throws if cause already set
throw wrapper;

// Accessing the chain:
try { callService(); }
catch (ServiceException e) {
    Throwable cause = e.getCause(); // direct cause
    while (cause != null) {
        System.out.println("Caused by: " + cause);
        cause = cause.getCause(); // walk full chain
    }
    e.printStackTrace(); // prints full chain with "Caused by:" blocks
}

// Anti-pattern: losing the cause
try { jdbcQuery(); }
catch (SQLException e) {
    throw new ServiceException("error"); // BAD: cause LOST – debug nightmare
    throw new ServiceException("error", e); // GOOD: cause preserved
}

// Exception translation pattern:
```

```
// Low-level: IOException, SQLException, JMSEException
// Mid-level: DataAccessException, MessagingException
// High-level: OrderProcessingException, PaymentException
// Each layer wraps with context; bottom of chain has root cause
```

Section 5 — Custom Exception Hierarchy

```
// Base domain exception – all application exceptions extend this
public class AppException extends RuntimeException {
    private final String errorCode;
    private final transient Object[] params;
    public AppException(String errorCode, String message) {
        super(message);
        this.errorCode = errorCode;
        this.params = new Object[0];
    }
    public AppException(String errorCode, String message, Throwable cause) {
        super(message, cause);
        this.errorCode = errorCode;
        this.params = new Object[0];
    }
    public String getErrorCode() { return errorCode; }
}

// Domain-specific exceptions
public class OrderNotFoundException extends AppException {
    private final Long orderId;
    public OrderNotFoundException(Long id) {
        super("ORDER_NOT_FOUND", "Order not found: " + id);
        this.orderId = id;
    }
    public Long getOrderId() { return orderId; }
}

public class InsufficientInventoryException extends AppException {
    private final String sku;
    private final int requested, available;
    public InsufficientInventoryException(String sku, int req, int avail) {
        super("INSUFFICIENT_INVENTORY",
            "Not enough stock for " + sku + ": requested=" + req + ", available=" + avail);
        this.sku = sku; this.requested = req; this.available = avail;
    }
}

public class PaymentDeclinedException extends AppException {
    public enum Reason { INSUFFICIENT_FUNDS, CARD_EXPIRED, FRAUD_DETECTED, NETWORK_ERROR }
    private final Reason reason;
    public PaymentDeclinedException(Reason reason, String message) {
        super("PAYMENT_DECLINED", message);
        this.reason = reason;
    }
    public Reason getReason() { return reason; }
}

// Usage:
try {
    orderService.placeOrder(request);
} catch (OrderNotFoundException e) {
    return ResponseEntity.notFound().build();
} catch (InsufficientInventoryException e) {
    return ResponseEntity.badRequest().body("Out of stock: " + e.getSku());
}
```

```
} catch (PaymentDeclinedException e) {  
    if (e.getReason() == NETWORK_ERROR) retryPayment();  
    else return ResponseEntity.paymentRequired().build();  
}
```


Section 6 — finally Block Semantics

6.1 When finally Runs — and When It Doesn't

Scenario	finally Runs?	Notes
Normal completion (no exception)	YES	Always
Exception thrown (caught)	YES	Runs after catch
Exception thrown (not caught)	YES	Runs before propagation
return in try block	YES	Finally runs, then returns
break/continue in try	YES	Finally runs before jump
System.exit() called	NO	JVM process exits immediately
JVM crash / kill -9	NO	OS kills the process
Infinite loop before finally	NO	Never reaches finally
Daemon thread + JVM exit	Maybe	JVM may abort daemon threads

6.2 Dangerous finally Patterns

```
// DANGER 1: return inside finally – suppresses original return AND exception
int dangerous() {
    try {
        return 1; // this return is SUPPRESSED
    } finally {
        return 2; // THIS return wins – always returns 2!
    }
}

// dangerous() always returns 2, even if try throws an exception
// DANGER 2: exception in finally suppresses original exception
void badFinally() throws Exception {
    try {
        throw new IOException("original"); // thrown first
    } finally {
        throw new RuntimeException("finally"); // REPLACES original – cause LOST
    }
}

// Only RuntimeException propagates; IOException is silently discarded
// Use TWR instead – TWR preserves original via suppressed exception mechanism
// CORRECT: lock release in finally (non-AutoCloseable resource)
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    // critical section
    updateSharedState();
} finally {
    lock.unlock(); // ALWAYS in finally – prevents lock leak on exception
}

// CORRECT: manual cleanup when TWR can't be used
InputStream in = null;
try {
    in = new FileInputStream(path);
    return readBytes(in);
} catch (IOException e) {
    throw new DataReadException("Failed reading: " + path, e);
} finally {
    // ... cleanup code ...
}
```

```

if (in != null) {
try { in.close(); } catch (IOException ignored) { /* best effort */ }
}
}

```

Section 7 — Exception Anti-Patterns

Anti-Pattern	Code Example	Why Bad	Fix
Swallowing	<code>catch(Exception e){}</code>	Hides bugs; silent failures	At minimum: <code>log.error(msg, e)</code>
Catch-all too broad	<code>catch(Exception e)</code>	Catches OOM, NPE etc.	Catch specific types
Using for flow control	<code>try{parseInt(s)}catch(NFE){}</code>	1000x slower than if/check	Use <code>Character.isDigit</code> or regex first
Losing cause	<code>throw new Ex(e.getMessage())</code>	Stack trace lost	<code>throw new Ex(msg, e)</code>
Double-logging	<code>log(e); throw e; (then caller logs)</code>	Duplicate log entries	Log OR re-throw, not both
Swallowing <code>InterruptedException</code>	<code>catch(IE e){}</code>	Breaks thread cancellation	<code>Thread.currentThread().interrupt()</code>
Catching Error	<code>catch(Error e){}</code>	JVM state may be corrupt	Let it propagate; maybe catch OOM with extreme care
Checking null with exception	<code>if(x==null) throw new Exception()</code>	Too broad; use specific type	<code>throw new NullPointerException(msg)</code> or <code>Objects.requireNonNull</code>
Declaring throws Exception	<code>void m() throws Exception</code>	Loses type information	Declare specific exception types
Re-throw without cause	<code>throw new Ex(e.getMessage())</code>	No stack trace from origin	<code>throw new Ex(msg, e)</code>

Section 8 — Production Exception Handling Patterns

8.1 Retry with Exponential Backoff

```
@FunctionalInterface
interface ThrowingSupplier<T> {
    T get() throws Exception;
}

public static <T> T withRetry(ThrowingSupplier<T> action,
    int maxAttempts, long baseDelayMs,
    Class<? extends Exception>... retryOn) throws Exception {
    int attempts = 0;
    while (true) {
        try {
            return action.get();
        } catch (Exception e) {
            attempts++;
            boolean isRetryable = Arrays.stream(retryOn)
                .anyMatch(cls -> cls.isInstance(e));
            if (!isRetryable || attempts >= maxAttempts) throw e;
            long delay = baseDelayMs * (1L << (attempts - 1)); // exponential: 100,200,400ms
            log.warn("Attempt {} failed, retrying in {}ms: {}", attempts, delay, e.getMessage());
            Thread.sleep(Math.min(delay, 30_000)); // cap at 30s
        }
    }
}

// Usage:
User user = withRetry(
    () -> userService.fetchFromRemote(id),
    3, 100,
    SocketTimeoutException.class, ConnectException.class
);
```

8.2 Global Exception Handler (Spring)

```
// Spring @ControllerAdvice – central exception-to-HTTP mapping
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(OrderNotFoundException e) {
        return ResponseEntity.status(404)
            .body(new ErrorResponse(e.getErrorCode(), e.getMessage()));
    }

    @ExceptionHandler(InsufficientInventoryException.class)
    public ResponseEntity<ErrorResponse> handleInventory(InsufficientInventoryException e) {
        return ResponseEntity.status(422)
            .body(new ErrorResponse(e.getErrorCode(), e.getMessage()));
    }

    @ExceptionHandler(Exception.class) // catch-all for unexpected
    public ResponseEntity<ErrorResponse> handleUnexpected(Exception e,
        HttpServletRequest req) {
        String correlationId = MDC.get("correlationId"); // from MDC
        log.error("Unhandled exception [{}]: {}", correlationId, req.getRequestURI(), e);
        return ResponseEntity.status(500)
            .body(new ErrorResponse("INTERNAL_ERROR", "An unexpected error occurred"));
    }

    // Never expose e.getMessage() to client for unexpected exceptions
}
```

```

}
}
// Thread.setDefaultUncaughtExceptionHandler – for non-web (background threads)
Thread.setDefaultUncaughtExceptionHandler((thread, throwable) -> {
log.error("Uncaught exception in thread {}: ", thread.getName(), throwable);
// alert monitoring system
});

```

8.3 Try-Result Functional Pattern

```

// Vavr-inspired Try<T> – encapsulate success or failure functionally
// Avoids checked exception propagation through functional pipelines
sealed interface Try<T> permits Try.Success, Try.Failure {
record Success<T>(T value) implements Try<T> {}
record Failure<T>(Throwable error) implements Try<T> {}
static <T> Try<T> of(Callable<T> supplier) {
try { return new Success<>(supplier.call()); }
catch (Throwable t) { return new Failure<>(t); }
}
default <R> Try<R> map(Function<T, R> fn) {
return switch (this) {
case Success<T> s -> Try.of(() -> fn.apply(s.value()));
case Failure<T> f -> new Failure<>(f.error());
};
}
default T getOrElse(T defaultValue) {
return switch (this) {
case Success<T> s -> s.value();
case Failure<T> f -> defaultValue;
};
}
default Try<T> recover(Function<Throwable, T> fn) {
return switch (this) {
case Success<T> s -> this;
case Failure<T> f -> Try.of(() -> fn.apply(f.error()));
};
}
}

// Usage – clean functional pipeline, no try/catch boilerplate:
String result = Try.of(() -> readFile(path))
.map(content -> parse(content))
.map(data -> data.getName())
.recover(err -> "default")
.getOrElse("fallback");

```

Interview Q&A; — 52 Questions

Purple [ADV] questions target senior/principal-level interviews. Standard questions suit mid-level Java engineers.

Part 1 — Hierarchy & Fundamentals

Q1. What is the difference between Error, checked Exception, and unchecked (Runtime) Exception?

Error: JVM-level fatal problems (OutOfMemoryError, StackOverflowError) — do not catch; JVM state may be corrupt. Checked Exception: extends Exception but not RuntimeException; compiler requires you to catch or declare throws; for recoverable external failures (file not found, DB down). Unchecked / RuntimeException: extends RuntimeException; no compiler requirement; for programming errors and violated preconditions (NullPointerException, ArrayIndexOutOfBoundsException).

Q2. What is the difference between throw and throws?

throw: statement that actually raises an exception at runtime — throw new IOException("msg"). throws: declaration on a method signature listing checked exceptions it may propagate — void readFile() throws IOException. throws is a promise to callers; throw is the act. Unchecked exceptions don't require throws declaration (but documenting them with @throws in Javadoc is still good practice for public APIs).

Q3. What is the difference between getMessage(), toString(), and printStackTrace()?

getMessage(): returns just the exception message string (may be null). toString(): returns className: message — e.g., java.io.IOException: File not found. printStackTrace(): prints the full stack trace to stderr — class, message, each frame. For production logging: log.error("context message", e) — logger handles formatting including stack trace based on log level; never use printStackTrace() in production.

Q4. Can a constructor throw an exception?

Yes. If a constructor throws, the object is not fully created — any finally blocks in the constructor run, but the object reference is never returned. Callers must handle or propagate the exception. Example: new FileInputStream(nonExistentPath) throws FileNotFoundException. TWR handles this correctly — if the resource constructor throws, no close() is called on that resource (it was never initialised).

Q5. What happens when an exception is thrown in a static initializer block?

A static initializer that throws a checked exception causes a compile error (static blocks can only throw unchecked). If it throws an unchecked exception, it is wrapped in ExceptionInInitializerError (a subclass of Error). Subsequent attempts to use the class throw NoClassDefFoundError. This makes static initializer failures very difficult to diagnose.

Q6. Can you catch multiple exception types in one catch block? Any restrictions?

Yes, Java 7+ multi-catch: catch(IOException | SQLException e). Restrictions: the types must be UNRELATED — if one is a superclass of another, the compiler rejects it (catch(IOException | FileNotFoundException e) is a compile error since FileNotFoundException extends IOException). The caught variable e is effectively final in a multi-catch block.

Q7. What is the correct order for multiple catch blocks?

More specific (subclass) BEFORE less specific (superclass). If a superclass catch appears first it would catch all subtypes, making the subclass catch unreachable — compile error. Example: FileNotFoundException first, then IOException, then Exception (if needed). Never put catch(Exception e) before more specific types unless it's a catch-all at the very end.

Q8. What is the difference between Throwable.initCause() and passing cause to the constructor?

Constructor: new AppException("msg", cause) — preferred; sets cause atomically at creation. initCause(Throwable t): sets cause after construction; can only be called ONCE; throws IllegalArgumentException if cause is the exception itself, and IllegalStateException if cause was already set. Use initCause() only when you need to set the cause after construction (e.g., when working with legacy exceptions that lack a cause constructor).

Part 2 — try-with-resources & finally

Q9. What is try-with-resources and what interface must the resource implement?

TWR (Java 7+) automatically closes resources declared in the try() clause after the block exits (normally or exceptionally). Resources must implement `AutoCloseable` (`close()` throws `Exception`) or its subinterface `Closeable` (`close()` throws `IOException`). Resources are closed in REVERSE order of declaration. TWR is far safer than try-finally because it correctly handles `close()` exceptions via suppression.

Q10. In what order are resources closed in a multi-resource TWR block?

Reverse order of declaration. If you declare: `try(A a = ...; B b = ...; C c = ...)`, they close as: `c.close()`, then `b.close()`, then `a.close()`. This mirrors stack-like LIFO semantics — the last resource opened is the first closed. Each `close()` exception is handled independently; all `close()` calls happen even if one throws.

Q11. What is a suppressed exception and how do you access it?

When the try body throws exception `X` and `close()` also throws exception `Y`, `X` is the primary exception (propagated) and `Y` is suppressed (attached to `X` via `Throwable.addSuppressed()`). Access with: `Throwable[] suppressed = e.getSuppressed()`; This preserves both exceptions — before Java 7 try-finally, the `close()` exception would silently replace the body exception, losing the original cause entirely.

Q12. When does finally NOT execute?

Four cases: 1) `System.exit()` is called — JVM process terminates immediately. 2) JVM crash or kill -9 — OS terminates the process. 3) The current thread enters an infinite loop or deadlock before reaching finally. 4) For daemon threads — JVM may not wait for them when all non-daemon threads finish. Note: `Thread.stop()` (deprecated) can also prevent finally by throwing `ThreadDeath` at any point.

Q13. What is the danger of returning inside a finally block?

A return in finally SUPPRESSES any return or exception from the try/catch blocks. The finally return always wins regardless of what happened before. Example: `try{return 1;}finally{return 2;}` always returns 2 — the return 1 is discarded. Even worse: `try{throw new Exception();}finally{return 2;}` — the exception is silently swallowed. Rule: NEVER use return, break, or continue inside a finally block.

Q14. What is the danger of throwing from a finally block?

An exception thrown from finally replaces any exception from the try/catch block — the original exception is silently lost (not even suppressed, unlike TWR). This is why finally should only contain cleanup code (close, unlock) with their own try/catch for close errors, not business logic that can throw. TWR handles this correctly via suppression; manual try-finally does not.

Q15. How do you properly release a lock in a finally block?

`lock.lock(); try { criticalSection(); } finally { lock.unlock(); }` The `lock()` call must be BEFORE the try block — if `lock()` itself throws, you don't want finally to call `unlock()` on a lock you never acquired. If `lock.lock()` is inside try and throws, finally runs and calls `unlock()` unnecessarily. This is the standard idiom — lock acquisition before try, unlock in finally.

Part 3 — Custom Exceptions & Exception Design**Q16. When should you create a custom exception?**

Create a custom exception when: 1) You need to carry additional context (`orderId`, `errorCode`, `sku`). 2) Callers need to handle your exception differently from general exceptions. 3) You want to express domain meaning (`OrderNotFoundException` is clearer than `RuntimeException`). 4) You need a hierarchy of related exceptions with a common base. Don't create custom exceptions for general errors that are already well-served by `IllegalArgumentException`, `IllegalStateException`, `NullPointerException`, etc.

Q17. Should you extend Exception or RuntimeException for custom exceptions?

Almost always `RuntimeException` for application-level exceptions. Reasons: 1) No boilerplate propagation in callers. 2) Clean API — callers handle what they choose. 3) Consistent with Spring, JPA, Hibernate (all use unchecked). 4) Easier API evolution — can add new unchecked subtypes without breaking callers. Use checked only for TRULY recoverable external failures where you specifically want to force callers to handle — file operations, network I/O in utility libraries.

Q18. What fields should a well-designed custom exception have?

1) serialVersionUID (for Serializable — required if the exception might cross JVM boundaries). 2) Domain context fields: orderId, userId, errorCode — final, with getters. 3) All 4 constructors for flexibility: (message), (message, cause), (cause), (message, cause, enableSuppression, writeableStackTrace). 4) The cause is the most important — always provide a constructor that takes Throwable cause. 5) Mark fields that can't be serialized as transient. Do NOT put mutable or non-serializable objects in exception fields if they cross JVM boundaries.

Q19. What is exception translation and when should you apply it?

Exception translation: catch a low-level exception and re-throw as a higher-level domain exception, preserving the cause chain. Apply when: lower-level exception details (SQL state, file path) are implementation details that callers shouldn't depend on; you want to add context; you want a consistent exception hierarchy across your API regardless of underlying technology. Example: `catch(SQLException e) { throw new DataAccessException("Failed to load user", e); }` Callers of your UserService don't need to know you use JDBC.

Q20. What is the serialVersionUID and why does it matter for exceptions?

serialVersionUID is a long field that identifies the class version for serialization. If it's not declared, Java computes it from class structure — if you change the class (add a field), the computed UID changes and deserialization of old instances fails with `InvalidClassException`. For exceptions: if your exception crosses JVM boundaries (RMI, Java serialization, some message queues), both sides must have compatible UIDs. Best practice: always declare private static final long serialVersionUID = 1L; in custom exceptions.

Part 4 — InterruptedException & Thread Safety**Q21. How do you correctly handle InterruptedException?**

NEVER swallow it: `catch(InterruptedException e) {}` — this permanently loses the interrupt signal. Option 1 (most common): re-interrupt the thread: `catch(IE e) { Thread.currentThread().interrupt(); }` This restores the interrupt flag so the calling code can detect it. Option 2: let it propagate — add throws `InterruptedException` to your method signature. Option 3: wrap in unchecked and re-interrupt: `catch(IE e) { Thread.currentThread().interrupt(); throw new TaskInterruptedException(e); }` `InterruptedException` is Java's cooperative cancellation mechanism — swallowing it breaks it.

Q22. What does Thread.interrupted() do differently from Thread.currentThread().isInterrupted()?

`Thread.interrupted()` (static): returns the interrupt status of the CURRENT thread AND CLEARS it.
`Thread.currentThread().isInterrupted()` (instance): returns interrupt status WITHOUT clearing it. When you catch `InterruptedException`, the flag is already cleared by the JVM. When you check the flag in a loop (for long computations), use `isInterrupted()` unless you want to consume the interrupt: `while(!Thread.currentThread().isInterrupted()) { doWork(); }`

Part 5 — Anti-Patterns & Best Practices**Q23. What is exception swallowing and why is it dangerous?**

Exception swallowing: `catch(Exception e) {}` — the exception is caught and nothing is done. Dangerous because: 1) Root cause is hidden; system appears to work but produces wrong results. 2) Hard to debug — no log, no trace, no error. 3) Resource leaks may go undetected. Minimum fix: `log.error("Unexpected error", e)` inside the catch. Better: re-throw as an unchecked exception with context. If you truly want to ignore an exception (rare), add an explicit comment: `// intentionally ignored`.

Q24. What is the problem with using exceptions for flow control?

1) Performance: creating an exception captures the full stack trace (`fillInStackTrace` native method) — 100-1000x slower than a normal conditional check. 2) Readability: exception-based flow is unexpected and hard to follow. 3) Intent mismatch: exceptions signal EXCEPTIONAL conditions; expected paths (invalid input, empty result) are not exceptional. Use Optional, null check, or return code for expected paths. Example: use `Character.isDigit` or regex before `parseInt`; use `Map.containsKey` before `get`; use `Files.exists` before `FileInputStream`.

Q25. What is double logging and how do you prevent it?

Double logging: logging an exception at the point it's caught AND also logging it again at a higher level — produces duplicate log entries making debugging confusing. Rule: LOG or RE-THROW, not both. If you log and handle (swallow), no re-throw needed. If you re-throw (possibly wrapped), do NOT log — let the top-level handler log once. Exception: you may log the point of origin with DEBUG level for trace, and log ERROR at the boundary. Log frameworks (SLF4J, Logback) support log levels to control this.

Q26. What is the difference between fail-fast and fail-safe in exception handling?

Fail-fast: detect and report errors immediately at the point of violation. Example: `Objects.requireNonNull(param, "param must not be null")` at method entry. Validates preconditions early, making bugs easier to diagnose. Fail-safe: attempt to continue operation despite errors, using defaults or fallbacks. Example: `catch(IOException e) { return defaultValue; }`. Choose fail-fast for programming errors (invalid state, null params); fail-safe for expected partial failures (one item in a batch fails, process rest).

Advanced Questions — Senior/Principal Level

These questions probe JVM internals, bytecode mechanics, concurrency, and production design. They appear in principal/staff engineer interviews.

Part 6 — JVM Internals & Bytecode

[ADV] Q27. How is try/catch/finally compiled to bytecode?

No special bytecode instructions — compiled using exception tables. Each try block has an exception table entry: [startPC, endPC) -> handlerPC, catchType. The JVM checks this table when an exception occurs. finally is compiled by DUPLICATING the finally code: once inline for normal exit and once for each exceptional exit (before re-throw). Java 6 and earlier used JSR/RET instructions for subroutines; Java 7+ uses duplication. You can view this with: `javap -c ClassName`.

[ADV] Q28. Why is creating a Throwable expensive and how can you skip stack trace capture?

Throwable() constructor calls fillInStackTrace() — a native method that walks the call stack and captures each frame (class, method, file, line). For deep stacks (hundreds of frames), this can take microseconds or milliseconds. To skip: `new RuntimeException(msg, cause, enableSuppression=true, writableStackTrace=false)` The 4-parameter constructor (Java 7+) lets you disable stack trace capture. Useful for exceptions used as flow control (circuit breaker, retry signals) where the stack trace adds no diagnostic value and performance matters.

[ADV] Q29. What is the @SneakyThrows trick in Lombok and how does it work under the hood?

@SneakyThrows lets you throw checked exceptions without declaring them in throws. It works via a generic type trick: the helper method is `private static void sneakyThrow(Throwable t) throws RuntimeException { throw (RuntimeException)(Throwable)t; }` But actually Lombok uses: `@SuppressWarnings("unchecked") static void sneakyThrow(Throwable t) throws T` — the compiler thinks T is RuntimeException due to erasure, so no checked exception warning. The JVM doesn't enforce checked exception declarations — they're purely a compiler check. Bytecode can throw any Throwable. Use sparingly — it deceives callers about what the method can throw.

[ADV] Q30. What are exception safety guarantees (from C++ applied to Java)?

Basic guarantee: if an operation fails, no resources are leaked and the object remains in some valid (though possibly different) state. Ensured by try-with-resources and finally for cleanup. Strong guarantee: if an operation fails, the state is exactly as it was before (atomicity). Achieved via defensive copies, transactional updates, or rollback on exception. No-throw guarantee: the operation never throws (e.g., `close()`, `unlock()`, `swap()`). Java's `AutoCloseable.close()` SHOULD be no-throw in practice but the API allows it. Design APIs with these guarantees in mind — document which level each method offers.

[ADV] Q31. How does Java handle exceptions in static initializers differently from constructors?

Static initializer blocks cannot declare checked exceptions — they would need a throws clause but initializer blocks have no signature. Any checked exception in a static block causes a compile error. Unchecked exceptions are wrapped in `ExceptionInInitializerError`. Once a static init fails, the class is marked failed. All subsequent uses throw `NoClassDefFoundError` (not `ExceptionInInitializerError` — it only fires once). This is a source of mysterious `NoClassDefFoundError` at runtime: always check if the root cause is `ExceptionInInitializerError` from a static initializer.

[ADV] Q32. What is the difference between OutOfMemoryError and StackOverflowError in terms of recovery?

`StackOverflowError`: thrown when the thread stack exceeds its limit (default 512KB-1MB, configurable with `-Xss`). Each caught `StackOverflowError` unwinds frames, so the JVM may recover after catching it — but the thread's stack is compromised. Catching is rarely useful. `OutOfMemoryError`: thrown when heap or metaspace is exhausted. Recovery is extremely risky — any operation may trigger another OOM. Possible (with extreme care): if you have a reserved emergency memory buffer that you release on OOM to allow graceful shutdown. Both are Errors — the JVM may be in an unknown state. The correct response is usually to log, alert, and restart the process.

[ADV] Q33. How does Spring's @Transactional determine when to rollback?

By default @Transactional rolls back on RuntimeException and Error (unchecked), but NOT on checked exceptions. This is why Spring uses unchecked exceptions throughout. Override: @Transactional(rollbackFor=IOException.class) — rollback on checked IOException too. @Transactional(noRollbackFor=BusinessRuleException.class) — don't rollback for a specific type. The decision is made in TransactionAspectSupport.completeTransactionAfterThrowing() via RollbackRuleAttribute matching — it walks the exception hierarchy to find the most specific matching rule.

[ADV] Q34. How do you implement a circuit breaker using exceptions?

Circuit breaker has 3 states: CLOSED (normal), OPEN (failing, reject fast), HALF-OPEN (test). Track failure count; if it exceeds threshold within a time window, open the circuit. When OPEN, throw CircuitOpenException immediately without calling the remote service. After a timeout, allow one probe call (HALF-OPEN); if it succeeds, close the circuit; if it fails, reopen. Implementation: use AtomicInteger for failureCount, AtomicReference for state, ScheduledExecutor for half-open transition. Libraries: Resilience4j CircuitBreaker, Hystrix (deprecated). Key exception insight: CircuitOpenException is thrown WITHOUT filling in the stack trace (writableStackTrace=false) since it's a control-flow signal, not a diagnostic exception.

[ADV] Q35. What is the difference between checked exception handling in Java vs Kotlin?

Java: compiler enforces checked exceptions; callers must handle or propagate; creates coupling between API layers. Kotlin: no checked exceptions at all — all exceptions are unchecked. Kotlin's @Throws annotation generates throws declarations for Java interoperability but the Kotlin compiler itself doesn't enforce handling. Kotlin's approach aligns with modern Java practice (Spring, Hibernate all use unchecked). The tradeoff: Kotlin loses the compile-time reminder to handle recoverable failures; Java provides explicit forcing but at the cost of boilerplate and API coupling.

[ADV] Q36. How do exception handling and Java's structured concurrency interact?

In StructuredTaskScope (Java 21 preview), when any forked task throws an exception, the scope can be configured to cancel all other tasks. ShutdownOnFailure policy: first exception fails the scope; subsequent exceptions are suppressed onto the first (similar to TWR suppression). scope.throwIfFailed() re-throws the first failure. This is fundamentally different from CompletableFuture where exceptions must be explicitly handled per future; structured concurrency ensures no exception is silently lost.

[ADV] Q37. Explain how catch block matching works for multi-level inheritance hierarchies.

JVM exception matching: starting from the first catch block, checks if the thrown exception IS-AN instance of the catch type (using instanceof semantics). The first matching catch wins. For the bytecode exception table, entries are checked in declaration order. Implication: class hierarchy determines which handler runs. If DataAccessException extends RuntimeException, a catch(RuntimeException) before catch(DataAccessException) will catch DataAccessException — compile error warns of this. The JVM re-throws if no handler matches — the exception propagates up the call stack until caught or the thread terminates printing the stack trace.

[ADV] Q38. How do you build exception handling for a reactive (Project Reactor) pipeline?

In Reactor: onErrorReturn(T fallback) — emit a default value; onErrorResume(Function) — switch to a fallback Publisher; onErrorMap(Function) — transform exception type (like exception translation); doOnError(Consumer) — side-effect (logging) without changing the flow; retry(n) — resubscribe up to n times; retryWhen(Retry) — configurable retry with exponential backoff (Retry.backoff(3, Duration)). Unlike imperative code, reactive exception handling is declarative and composable. Key: exceptions must be caught within the reactive pipeline — throwing imperatively inside operators causes unhandled error emissions, not exceptions.

[ADV] Q39. What is the impact of exception handling on JIT compilation and performance?

Happy path: JIT compiles try block normally — no overhead when no exception occurs. Exception tables are metadata; the JIT only looks at them when an exception fires. Throwing: expensive due to stack trace capture (fillInStackTrace). JIT may deoptimize a method after frequent exceptions (HotSpot: rare case — exceptions are assumed exceptional). Exception-as-control-flow: if a method throws frequently, the JIT compiler may not inline it aggressively and may insert counter-checks. At scale, exceptions-as-flow can degrade throughput significantly. Pre-check with Optional/null/conditional instead.

[ADV] Q40. What is the checked exception controversy in API design, and what does Effective Java recommend?

Controversy: checked exceptions force callers to handle failures explicitly (good for safety) but create API coupling, boilerplate, and make lambdas/streams awkward (no throws in functional interfaces). James Gosling intended checked exceptions for recoverable conditions; in practice most checked exceptions aren't recoverable at the call site. Effective Java Item 70: use checked for truly recoverable conditions; item 71: avoid unnecessary checked exceptions (can you reasonably recover? if not, use unchecked). Item 77: never ignore exceptions. Modern consensus: unchecked with good documentation is preferred for most application code; checked justified for I/O libraries.

Exercises

Part A — Coding Challenges

- A1.** Implement a generic retry utility: `withRetry(ThrowingSupplier supplier, int maxAttempts, long baseDelayMs, Class... retryOn)`. Support exponential backoff capped at 30 seconds. Re-throw the last exception if all attempts fail.
- A2.** Design a complete custom exception hierarchy for an order management system: `AppException` (base), `OrderException`, `PaymentException` (with `Reason` enum), `InventoryException` (with `sku`, `requested`, `available` fields). Include `serialVersionUID`, proper constructors, and getters.
- A3.** Implement a safe resource wrapper: wrap any `AutoCloseable` with `try-with-resources`, and if `close()` throws, add it as suppressed to the primary exception using `addSuppressed`. Then verify it works correctly using a custom buggy `AutoCloseable`.
- A4.** Build a functional Try monad (sealed `Success/Failure` records): `Try.of(Callable)`, `map()`, `flatMap()`, `recover()`, `orElse()`, `getOrThrow()`. Write a pipeline that reads a file, parses JSON, extracts a field, and falls back to a default with zero try/catch blocks at the call site.
- A5.** Implement an `InterruptedException`-safe sleep utility: `safeSleep(long millis)` that re-interrupts the thread if interrupted during sleep, and a `countDown` method that processes N items with interruption checkpoints.

Part B — Debug & Fix

B1 – InterruptedException Swallowed

```
void waitForResult() {
    try {
        Thread.sleep(5000);
    } catch (InterruptedException e) {
        // ignore
    }
}
```

Fix: Swallowing `InterruptedException` destroys cooperative thread cancellation. Fix: `catch(InterruptedException e) { Thread.currentThread().interrupt(); }` This restores the interrupt flag so callers can detect the interruption.

B2 – return Inside finally Suppresses Exception

```
String loadConfig() {
    try {
        throw new IOException("config missing");
    } finally {
        return "default"; // silently swallows IOException!
    }
}
```

Fix: `return` in `finally` discards the `IOException` silently. Fix: remove `return` from `finally`; use `try-with-resources` or explicit `catch` to return default only for expected failures.

B3 – Exception Cause Lost

```
try {
    return jdbcTemplate.query(sql, mapper);
} catch (DataAccessException e) {
    throw new ServiceException(e.getMessage()); // cause LOST
}
```

Fix: Passing `e.getMessage()` loses the original stack trace. Fix: `throw new ServiceException("Failed to query users", e);` — always pass the cause `Throwable`.

B4 – Catching Exception Too Broadly

```
try {
    processOrder(order);
} catch (Exception e) { // catches OOM, NPE, everything
    log.warn("Minor issue: " + e.getMessage());
}
```

Fix: Catching Exception swallows OutOfMemoryError, NullPointerException, and other serious problems as 'minor'. Fix: catch specific expected types (OrderException, PaymentException). Have a final catch(Exception e) that logs as ERROR and re-throws or wraps as unchecked.

Part C — MCQ / Predict the Output

MC1. What is the output? try { return 1; } finally { return 2; }

- A) 1
- B) 2
- C) Both 1 and 2
- D) Compile error

Answer: B — finally return always wins and suppresses the try return. Method always returns 2.

MC2. A method declares throws IOException but only has a catch(IOException e) inside. Does the caller need to handle IOException?

- A) Yes — throws declaration requires callers to handle
- B) No — it's caught internally so it cannot propagate
- C) Compile error — can't declare throws if caught internally
- D) Depends on Java version

Answer: A — The compiler only sees the throws declaration — it trusts it. Even if the exception is always caught, the declaration is a contract obligation the caller must honor.

MC3. What happens if close() throws in a try-with-resources when the body also threw?

- A) Both exceptions are lost
- B) Only the close() exception propagates
- C) Body exception propagates; close() exception becomes suppressed
- D) JVM throws both simultaneously

Answer: C — TWR: body exception is primary; close() exception is attached via addSuppressed. Both are preserved.

MC4. Which correctly handles a checked exception in a lambda?

- A) list.forEach(s -> Files.readAllBytes(s));
- B) list.forEach(s -> { try { Files.readAllBytes(s); } catch(IOException e) { throw new RuntimeException(e); } });
- C) list.forEach((String s) throws IOException -> Files.readAllBytes(s));
- D) Both A and C

Answer: B — Functional interfaces don't declare checked exceptions. Must wrap in unchecked inside the lambda. A and C don't compile. B is the correct wrapping pattern.

MC5. static int x = 1 / 0; inside a class. What exception occurs when you first use the class?

- A) ArithmeticException
- B) ExceptionInInitializerError
- C) NoClassDefFoundError
- D) B on first use, C on subsequent uses

Answer: D — First use wraps the ArithmeticException in ExceptionInInitializerError. Any subsequent attempt to use the class throws NoClassDefFoundError.

MC6. Which catch block is unreachable?

- A) `catch(RuntimeException e)` before `catch(NullPointerException e)`
- B) `catch(IOException e)` before `catch(FileNotFoundException e)`
- C) `catch(Exception e)` before `catch(RuntimeException e)`
- D) All of the above

Answer: D — In all three cases, the first (supertype) catch would intercept everything the second (subtype) catch handles — making the second unreachable. Java compiler reports this as an error.

Quick Reference Cheat Sheet

Topic	Key Points to Remember
Error	JVM-level (OOM, StackOverflow) — DO NOT catch; JVM state may be corrupt
Checked	Recoverable external failures; compiler enforces handling; extends Exception (not Runtime)
Unchecked	Programming errors; extends RuntimeException; no compiler enforcement
throw vs throws	throw: raises the exception; throws: declares it on the method signature
Multi-catch	catch(A B e) — types must be UNRELATED; e is effectively final
Catch order	Subclass BEFORE superclass; else compile error (unreachable catch block)
TWR	Implements AutoCloseable; closed in REVERSE order; primary exception wins over close()
Suppressed	Body exception = primary; close() exception = suppressed via addSuppressed()
finally	Always runs (except System.exit, JVM crash, infinite loop, daemon thread abort)
finally return	NEVER return in finally — suppresses try return AND exceptions silently
finally throws	Replaces try exception (unlike TWR suppression) — use TWR to avoid this
Exception chaining	Always pass cause: new Ex(msg, cause) — never new Ex(e.getMessage())
InterruptedException	Never swallow; always Thread.currentThread().interrupt() to restore flag
Custom exceptions	Extend RuntimeException; add serialVersionUID; include domain context fields
Translation pattern	catch low-level (SQL/IO); wrap in domain exception; preserve cause chain
Anti-patterns	Swallowing, catch-all Exception, flow-control, double-logging, losing cause
Performance	Throwable captures stack trace (native) — 100-1000x slower; avoid in hot paths
Spring @Transactional	Rolls back on RuntimeException+Error; NOT checked; use rollbackFor to override

End of Sub-Chapter 1.8 — Exception Handling

Next: Sub-Chapter 1.9 — Java 8 Features